



ADVANCED DATABASE SYSTEMS

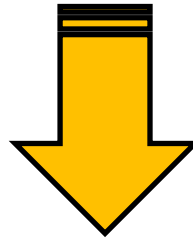
WEEK 6

QUERY OPTIMIZATION PART II



Query Optimization

For a given SQL Query



Goal:

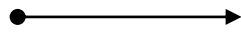
Build the most “optimal” execution plan for the query



Query Optimization

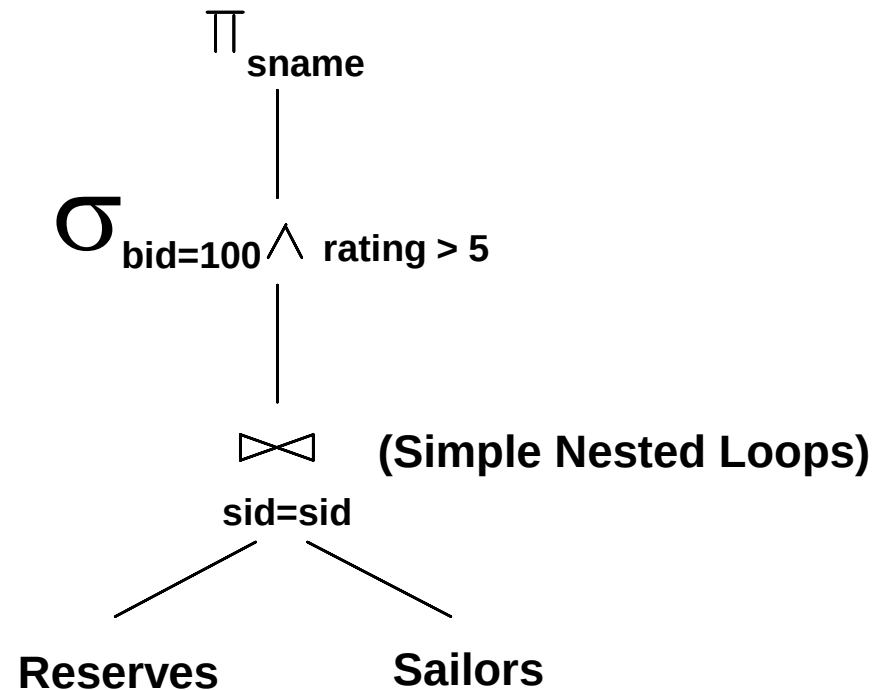
Goal:

Declarative SQL query



Imperative query execution plan:

```
SELECT S.sname
FROM Reserves R, Sailors S
WHERE R.sid = S.sid
      AND R.bid = 100
      AND S.rating > 5
```

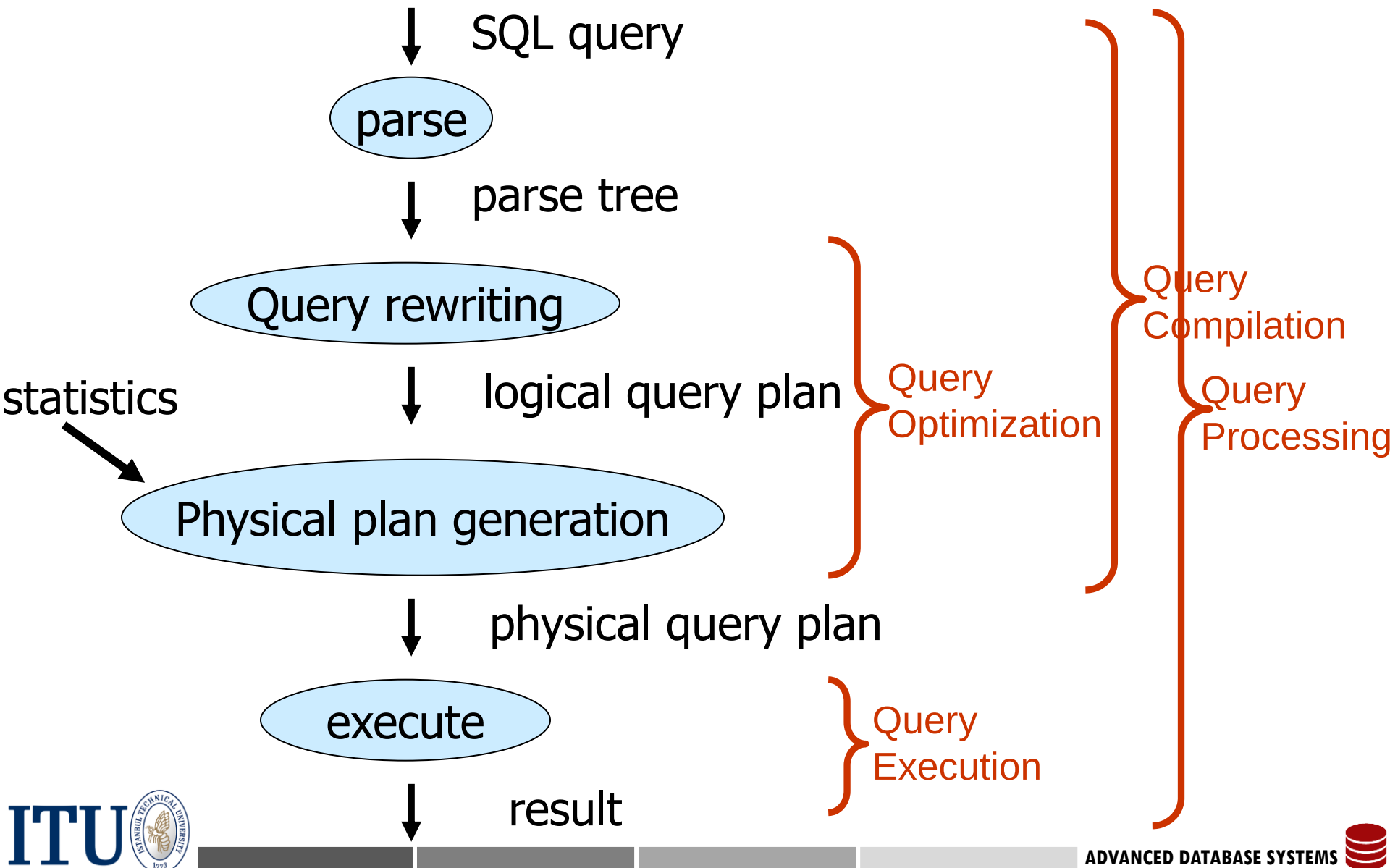


Plan: Tree of R.A. ops, with choice of alg for each op.

Ideally: Want to find best plan. **Practically:** Avoid worst plans!

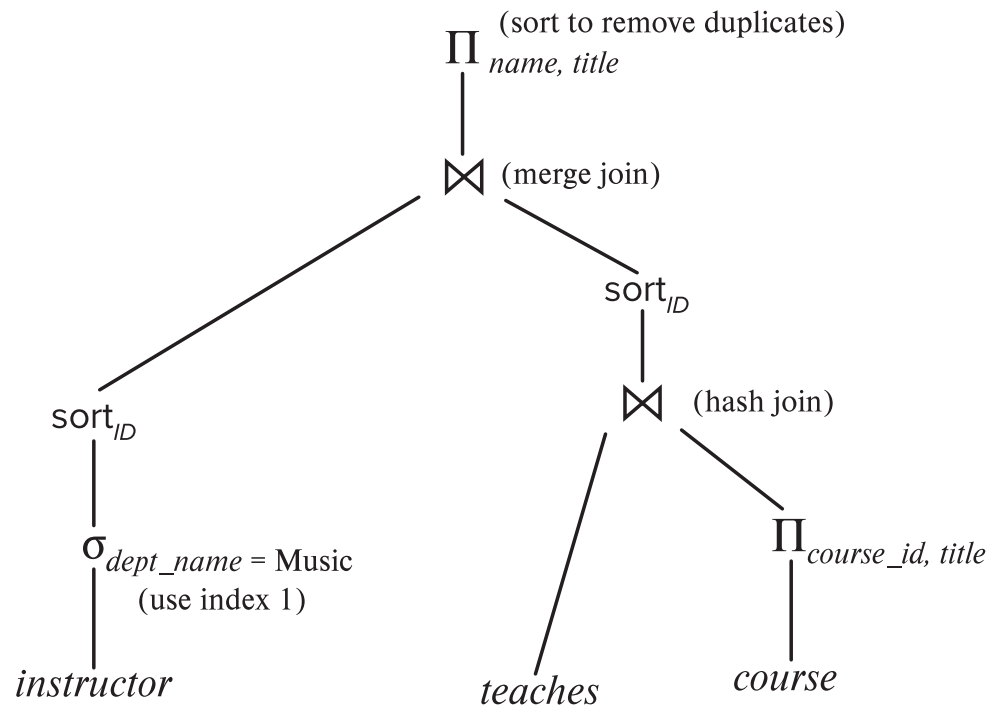


Overview of Query Processing



Introduction (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.

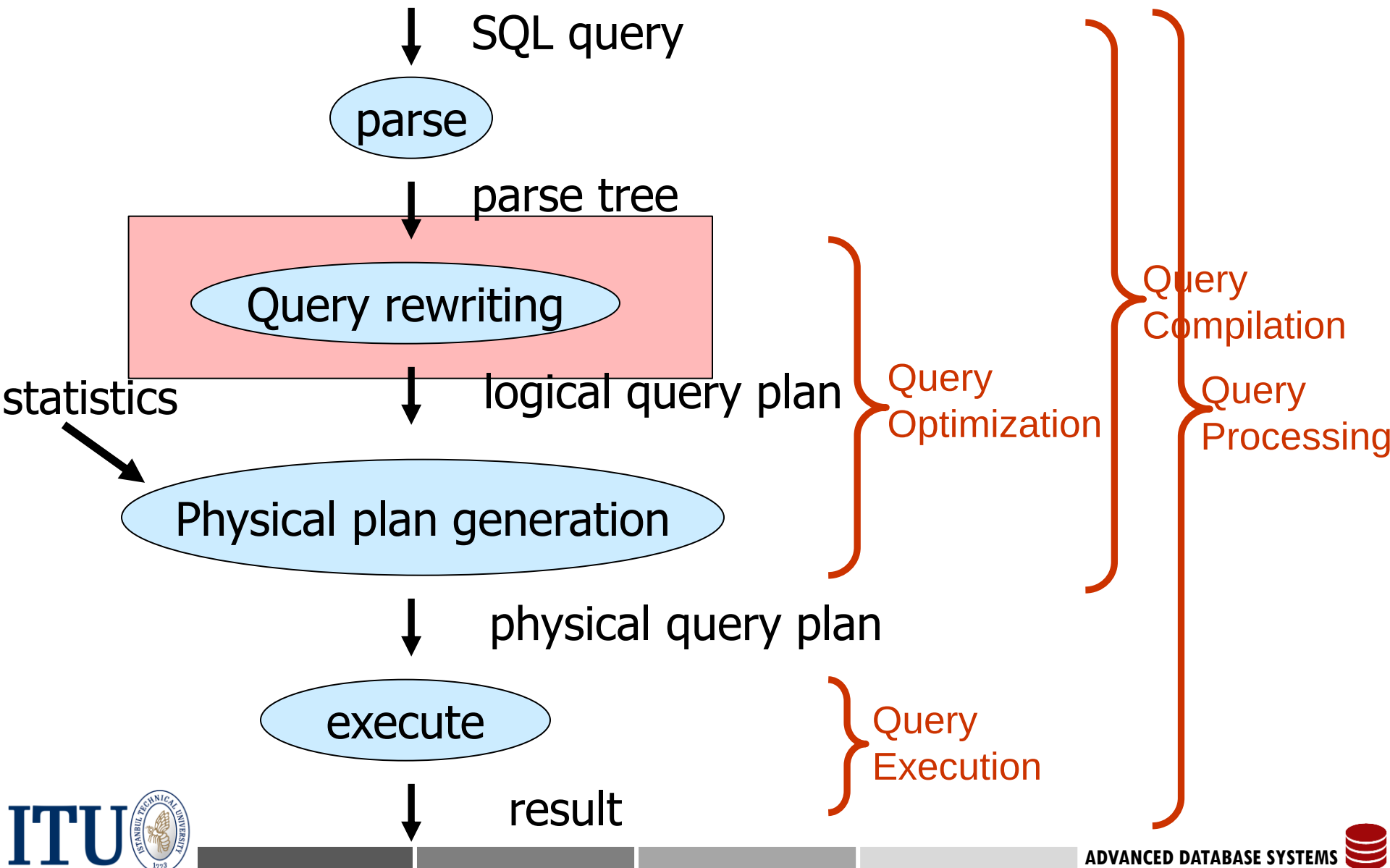


Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - e.g., seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 - ▶ Query rewriting:
 - transformations from one SQL query to another one using semantic properties.
 2. Annotate resultant expressions to get alternative query plans
Selecting query execution plan:
 - ▶ done on single query blocks (i.e., S-P-J blocks)
 - ▶ main step: join enumeration
 3. Choose the cheapest plan based on **estimated cost**
 - ▶ Statistical information about relations. Examples:
 - number of tuples, number of distinct values for an attribute
 - ▶ Statistics estimation for intermediate results
 - to compute cost of complex expressions
 - ▶ Cost formula for algorithms, computed using statistics



Overview of Query Processing



Generating Equivalent Expressions



Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if
 - they generate the same set of tuples on every *legal* database instance
 - **Note:** order of tuples is irrelevant
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if
 - ▶ the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice versa



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

Example: Imagine you are looking for employees who work in "Sales" **and** earn more than **\$50,000**.

- **Initial Plan:** Scan the entire Employees table and check both conditions for every single row.
- **Alternative Plan:** First, filter for Dept = 'Sales'. If that significantly reduces the number of rows, the second filter (Salary > 50000) has much less work to do.



Equivalence Rules

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Example: Think of a library database where you want books that are "Sci-Fi" and "Available."

- Scenario A: You filter all 1,000,000 books for "Available" first (maybe 500,000 books), then filter those for "Sci-Fi."
- "Scenario B: You filter for "Sci-Fi" first (maybe only 5,000 books), then check which of those are "Available."

Optimization: Rule 2 allows the database to pick Scenario B because checking 5,000 rows is much faster than checking 500,000.



Equivalence Rules

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E) \text{ where } L_1 \subseteq L_2 \dots \subseteq L_n$$

Example: Imagine a query that asks for a list of EmployeeNames from a subquery that originally pulled EmployeeNames, IDs, and Birthdays.

- Initial Plan: The inner "box" creates a temporary table with three columns, then the outer "box" throws two of them away.
- Optimized Plan: The optimizer realizes it only ever needs EmployeeNames. It instructs the database to ignore IDs and Birthdays from the very start, saving memory and processing time.



Equivalence Rules

4. Selections can be combined with Cartesian products and theta joins.

a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

This is the most critical rule for performance. It's the difference between a query taking seconds or hours.

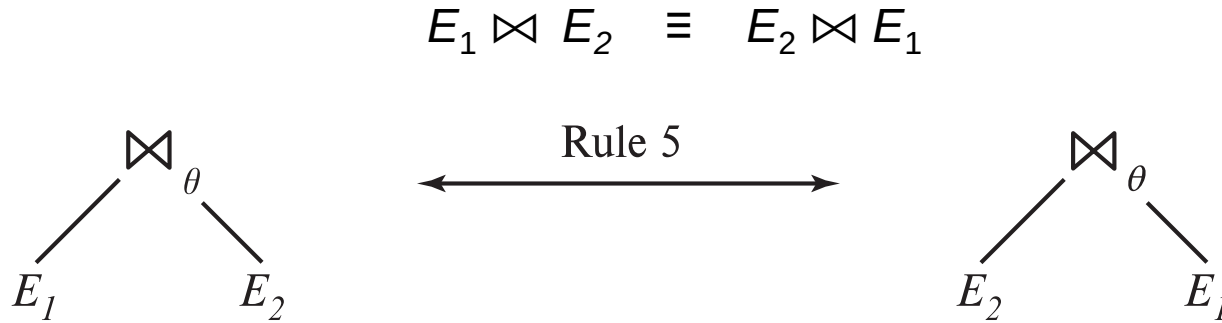
Example: Rule 4a (The "Cartesian Product" Trap): Suppose you want to join Students and Grades on StudentID.

- ❑ Bad Plan: Create a Cartesian Product ($E_1 \times E_2$). If you have 1,000 students and 1,000 grades, this creates 1,000,000 rows, then filters them.
- ❑ Good Plan: Use a Theta Join ($E_1 \bowtie_{\theta} E_2$). The database matches IDs as it reads the data, never creating that massive 1,000,000-row intermediate table.
- ❑ Rule 4b (Adding Conditions to Joins): If you are joining Orders and Customers and then filtering for $\text{OrderDate} > '2023-01-01'$, the optimizer uses this rule to fold that date filter directly into the join logic.



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

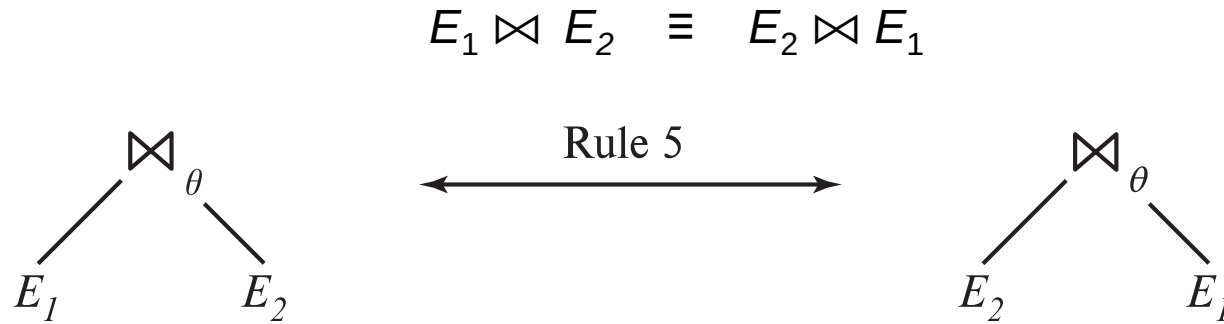


Example: Imagine you want to join a table of Countries (about 200 rows) with a table of Citizens (about 8 billion rows) on a CountryID.

- ❑ Plan A ($E_{\text{citizens}} \bowtie E_{\text{countries}}$): If the database uses a "Nested Loop Join," it might pick up a Citizen, then scan the Countries table to find a match. It repeats this 8 billion times.
- ❑ Plan B ($E_{\text{countries}} \bowtie E_{\text{citizens}}$): The database loads the 200 Countries into a fast memory "Hash Map" (small and easy to manage). Then, it streams the 8 billion Citizens through that map.

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.



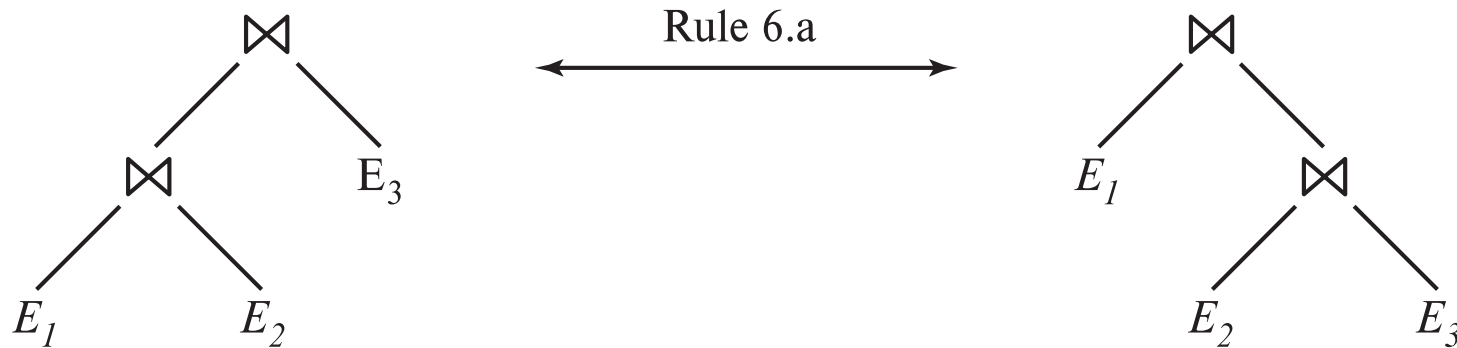
Comparison Summary

Feature	$E_1 \bowtie E_2$	$E_2 \bowtie E_1$
Logical Result	Identical	Identical
Execution Order	E_1 is processed first	E_2 is processed first
Optimizer Goal	Use if E_1 is smaller/better indexed	Use if E_2 is smaller/better indexed

Equivalence Rules (Cont.)

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$



This rule is vital when joining three tables, such as Customers, Orders, and Order_Items.

Example: Imagine the following data sizes:

- ❑ Customers (E_1): 1,000 rows,
- ❑ Orders (E_2): 1,000,000 rows,
- ❑ Order_Items (E_3): 10,000,000 rows.



Equivalence Rules (Cont.)

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

Scenario A: $(E_1 \bowtie E_2) \bowtie E_3$

The database first joins Customers and Orders. This creates an intermediate result of 1,000,000 rows (all orders matched to their customer names). Then, it joins those 1 million rows with the 10 million Order_Items.

Scenario B: $E_1 \bowtie (E_2 \bowtie E_3)$

The database first joins Orders and Order_Items. This creates a massive intermediate result of 10,000,000 rows. Then, it has to join that huge 10-million-row set with Customers.

Optimizer Goal: Keep the "working set" as small as possible. Large intermediate joins can spill out of RAM and onto the hard drive (disk), which is significantly slower. Rule 6 allows the optimizer to find a path that stays within the high-speed memory.



Equivalence Rules (Cont.)

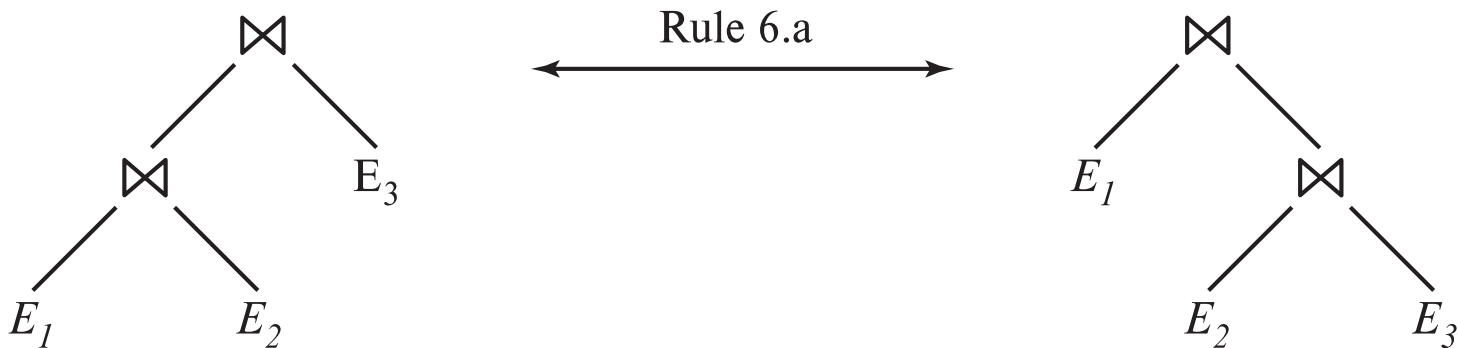
6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

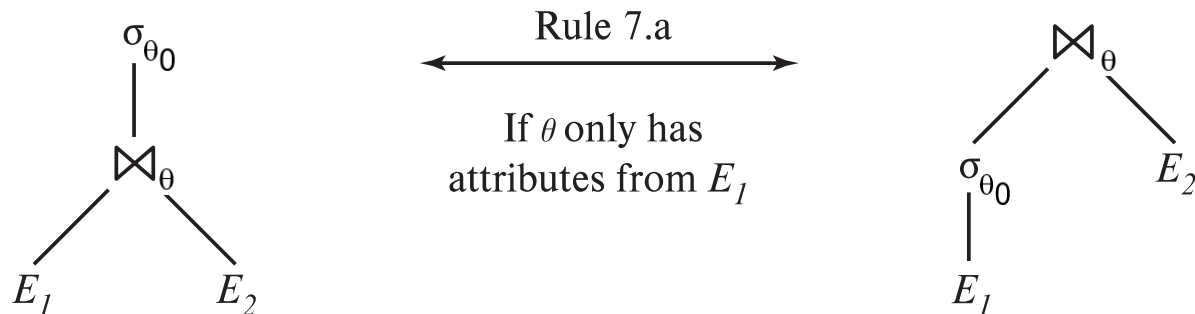


Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

similarly: $\sigma_{\theta_0 \wedge \theta}(E_1 \times E_2) \equiv \sigma_{\theta}((\sigma_{\theta_0}(E_1)) \times E_2)$



- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$



Equivalence Rules

Example: Imagine you have a Customers table (1 Million rows) and an Orders table (50 Million rows). You want to find all orders for customers living in "Iceland".

- ❑ **Scenario 1:** Join all 1M customers with all 50M orders. This creates a massive intermediate table (50M rows). Then, scan those 50M rows to find the ones where the country is "Iceland."
 - ❑ Result: Extremely slow; uses massive amounts of RAM/Disk.
- ❑ **Scenario 2:** Filter Customers first. If there are only 5,000 customers in Iceland, you now have a tiny list.
 - ❑ Then, join only those 5,000 customers with the Orders table.
 - ▶ Result: Lightning fast; the database only looks at a fraction of the data.



Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$$

(b) In general, consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- Let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$

Similar equivalences hold for outerjoin operations: $\bowtie$, $\ltimes$, and $\ltimes$

Equivalence Rules (Cont.)

- Imagine you are joining a
 - Products table (containing descriptions, photos, weights, dimensions) and a
 - Sales table (containing transaction IDs, credit card info, shipping addresses).
 - You just want to know the Product Name and the Total Quantity Sold.
- **Initial Plan** ($\Pi_{Name,Qty}(Products \bowtie Sales)$):
 - Join the entire Products table with the entire Sales table.
 - Every intermediate row contains photos, weights, credit card info, and shipping addresses.
 - Then, at the very end, discard everything except Name and Qty.
 - Problem: You likely ran out of RAM or hit the disk because the rows were too fat.

Equivalence Rules (Cont.)

- Imagine you are joining a
 - Products table (containing descriptions, photos, weights, dimensions) and a
 - Sales table (containing transaction IDs, credit card info, shipping addresses).
 - You just want to know the Product Name and the Total Quantity Sold.
- **Initial Plan** ($\Pi_{Name,Qty}(Products \bowtie Sales)$):
- **Optimized Plan** ($\Pi_{Name}(Products) \bowtie \Pi_{Qty}(Sales)$):
 - "Slim down" Products to only the Name (and the Join Key).
 - "Slim down" Sales to only the Qty (and the Join Key).
 - Then, join these two skinny tables.
 - Result: The join happens entirely in memory because the rows are tiny. It is significantly faster.

Equivalence Rules (Cont.)

- Rule 8(b) adds a small but vital "gotcha": The Join Keys.
 - If you are joining on ProductID, you can't throw that column away before the join, even if it's not in your final SELECT list.
 - The Logic: You push down the projection of all the columns you want PLUS any columns required to perform the join.
 - **Example:** If you want just the EmployeeName from a join of Employees and Departments on DeptID:
 - ▶ You push down a projection for (EmployeeName, DeptID).
 - ▶ You perform the join.
 - ▶ You perform a final projection to drop the DeptID.

Rule	Operation	Motivation	Benefit
Rule 7	Selection (σ)	Filter rows early.	Reduces the count of rows to join.
Rule 8	Projection (Π)	Filter columns early.	Reduces the width of rows to join.



Case Study

- **The Goal:** Find the names of employees in the "Research" department who were hired after 2020 and work on a project named "Alpha."
- **The SQL:**

```
SELECT E.name
FROM Employees E, Departments D, Projects P
WHERE E.dept_id = D.id
      AND E.id = P.emp_id
      AND D.dept_name = 'Research'
      AND E.hire_date > '2020-01-01'
      AND P.proj_name = 'Alpha';
```



Case Study – Cont'd.

Step 1: The "Naïve" Plan (Rule 4a)

- Initially, a computer might interpret this as a massive Cartesian Product ($E \times D \times P$) followed by a giant filter.
- The Problem:
 - If you have 1,000 employees, 10 departments, and 500 projects, the Cartesian product is 5 million rows before you even start filtering.
- Optimization:
 - Rule 4a converts these products into Theta Joins, so we only combine rows that actually match on IDs.

4. Selections can be combined with Cartesian products and theta joins.
a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$



Case Study – Cont'd.

Step 2: Selection Pushdown (Rule 7)

- Now we have joins, but joining all employees to all projects is still heavy.
- Rule 7 Application: We "push" the filters down to the base tables.
 - Filter Departments for dept_name = 'Research' first.
 - Filter Employees for hire_date > '2020' first.
 - Filter Projects for proj_name = 'Alpha' first.
 - Result:
 - ▶ Instead of joining 1,000 employees, we might only be joining the 50 who were hired recently.

7. The selection operation distributes over the theta join operation under the following conditions:
(a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$



Case Study – Cont'd.

Step 3: Join Reordering (Rules 5 & 6)

- We have three tables to join. In what order?
- Rule 5 & 6 Application: The optimizer calculates the “cost.”
 - Joining Departments (now 1 row: Research) with Employees (50 rows) results in a tiny intermediate set of 50 rows.
 - If we joined Employees with Projects first, we might have an intermediate set of 500 rows.
 - Decision:
 - ▶ The optimizer uses Associativity to group (Dept **JOIN** Emp) first, then join the result with Projects.

5. Join operations are commutative.

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

6. Join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$



Case Study – Cont'd.

Step 4: Projection Pushdown (Rule 8)

- Even though our tables have many columns (Address, Salary, Project Budget, etc.), our final SELECT only wants E.name.
- Rule 8 Application:
 - As we read from the disk, we throw away every column except the name and the IDs needed for the joins.
 - Result: The data moving through the "pipes" of the query plan is now very "thin," saving massive amounts of memory.

8. In general, consider a join $E_1 \bowtie_{\theta} E_2$.

Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.

Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and

Let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$



Case Study – Cont'd.

The Final "Optimized" Plan

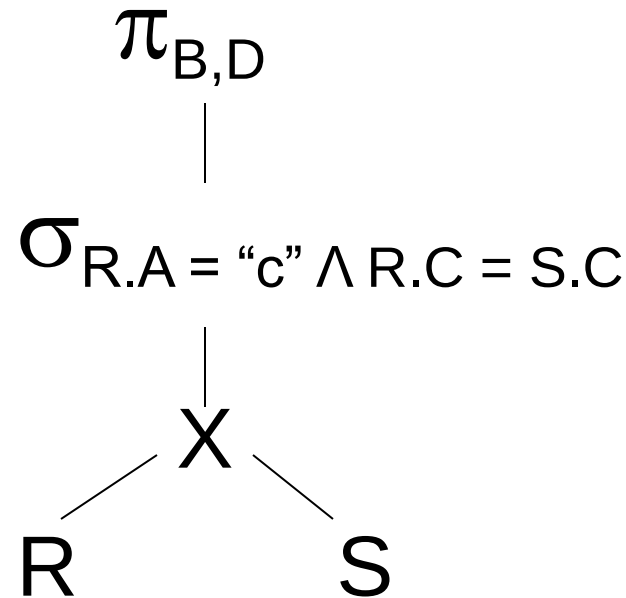
- By the time the database actually starts running, it isn't looking at millions of rows.
- It's looking at:
 - One specific department.
 - A few dozen recent employees.
 - One specific project.
 - Only the "Name" column.

This transformation reduces the work required from millions of operations to just a few hundred.



Query Transformation – Example 1

Select B,D
From R,S
Where R.A = "c" $\wedge$
R.C=S.C

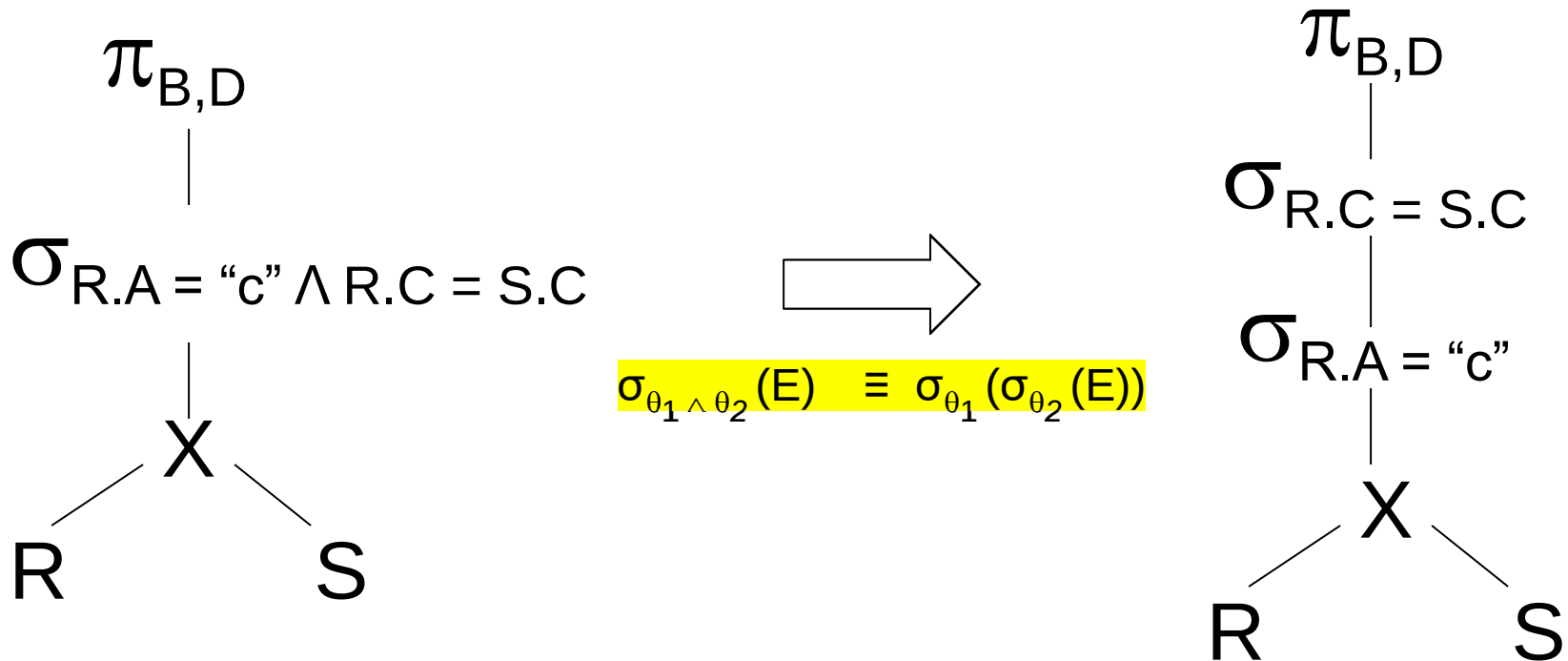


Initial Logical Plan

Relational Algebra: $\Pi_{B,D} [\sigma_{R.A=\text{"c"} \wedge R.C = S.C} (RXS)]$



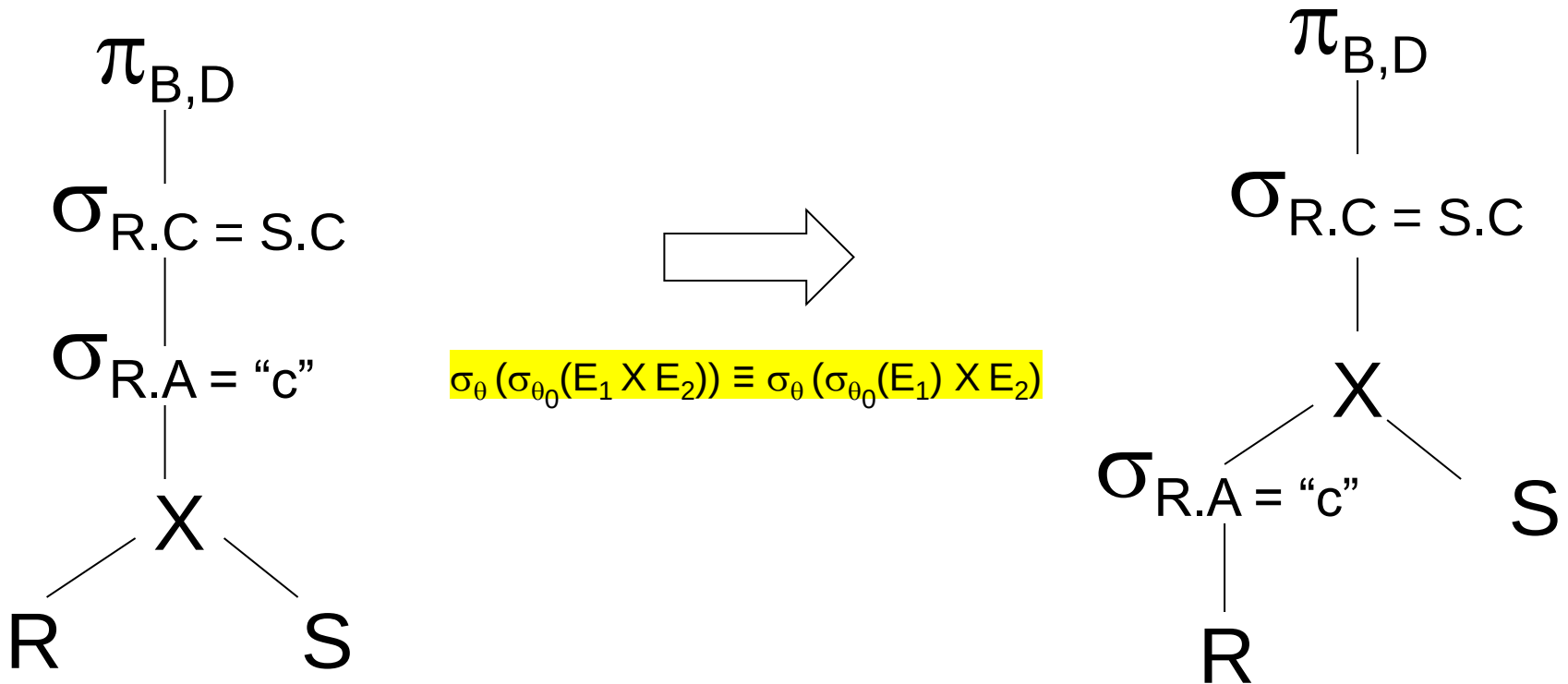
Apply Rewrite Rule (Step 1)



$$\Pi_{B,D} [\sigma_{R.C=S.C} [\sigma_{R.A="c"}(R \times S)]]$$

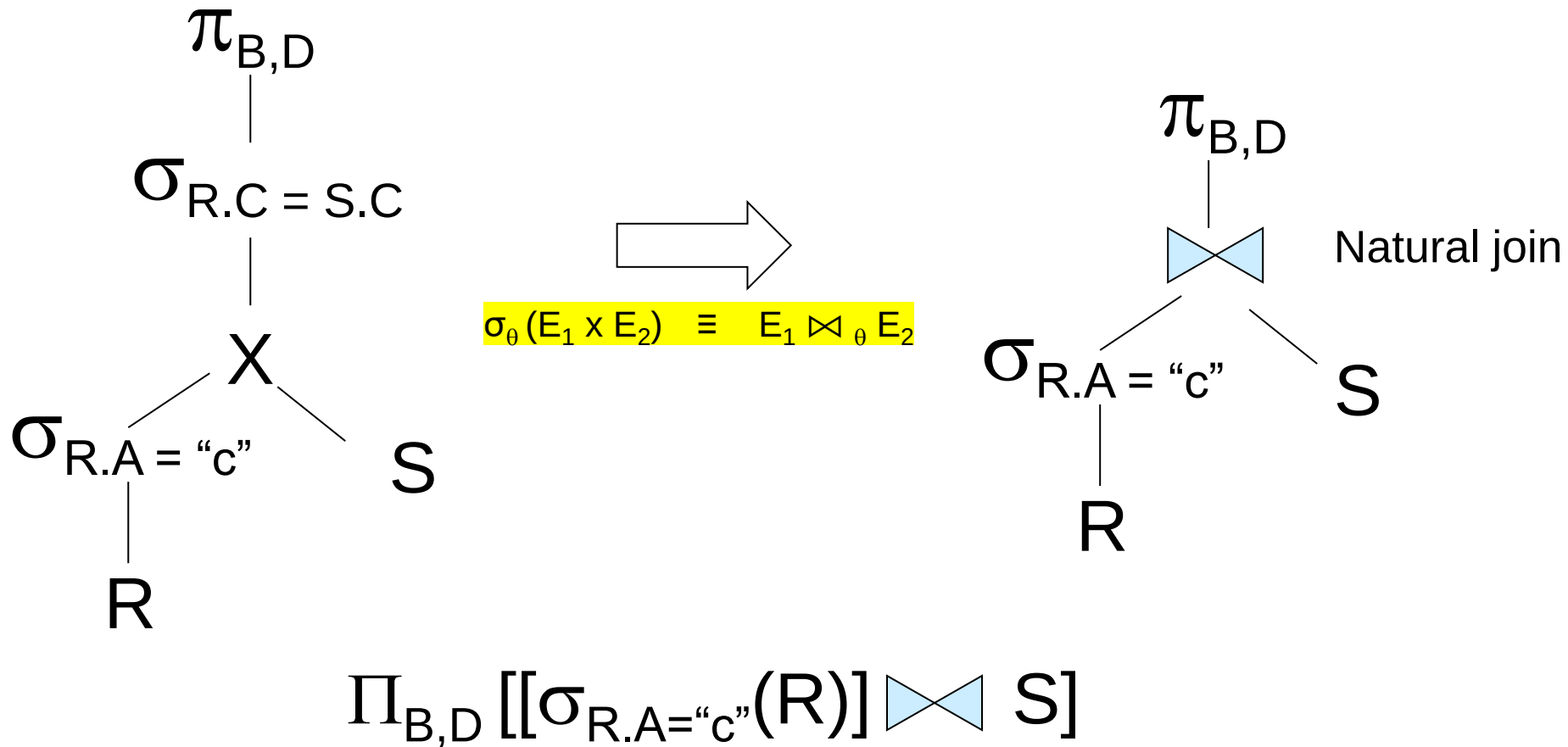


Apply Rewrite Rule (Step 2)



$$\Pi_{B,D} [\sigma_{R.C=S.C} [\sigma_{R.A="c"}(R)] \times S]$$

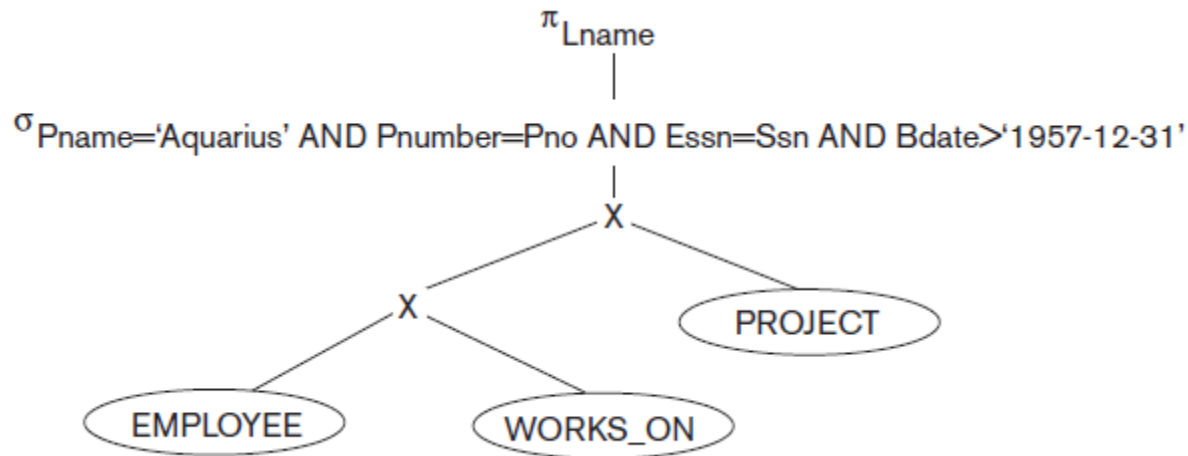
Apply Rewrite Rule (Step 3)



Query Transformation - Example 2

Q: SELECT E.Lname
 FROM EMPLOYEE E, WORKS_ON W, PROJECT P
 WHERE P.Pname='Aquarius' AND P.Pnumber=W.Pno AND E.Essn=W.Ssn
 AND E.Bdate > '1957-12-31';

(a)

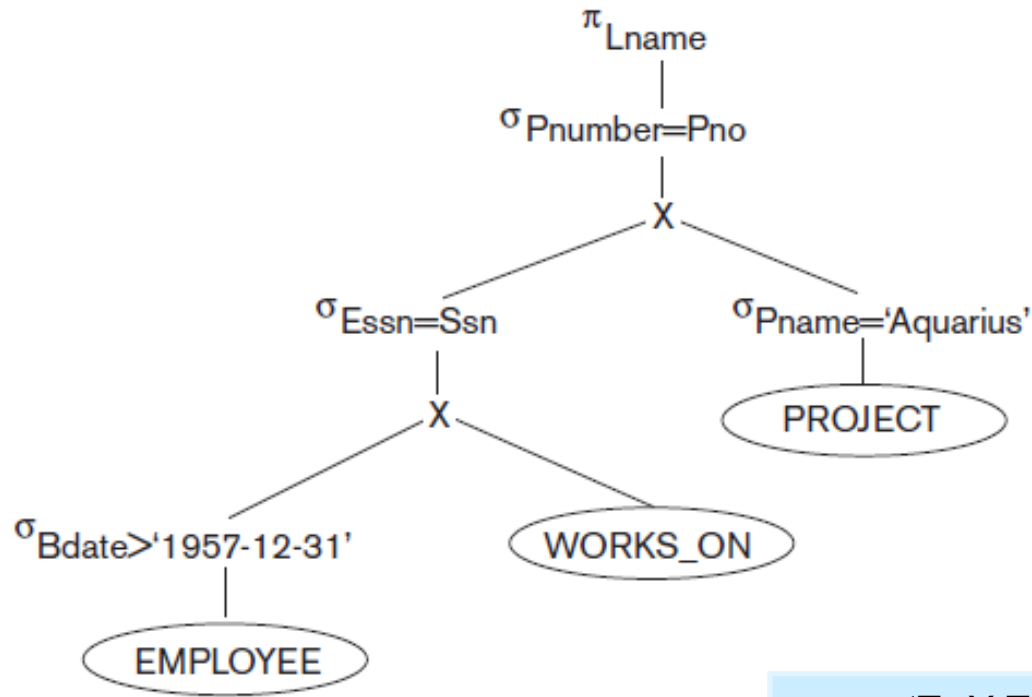


Steps in converting a query tree during heuristic optimization. (a) Initial (canonical) query tree for SQL query Q.



Query Transformation - Example 2 (cont'd.)

(b)

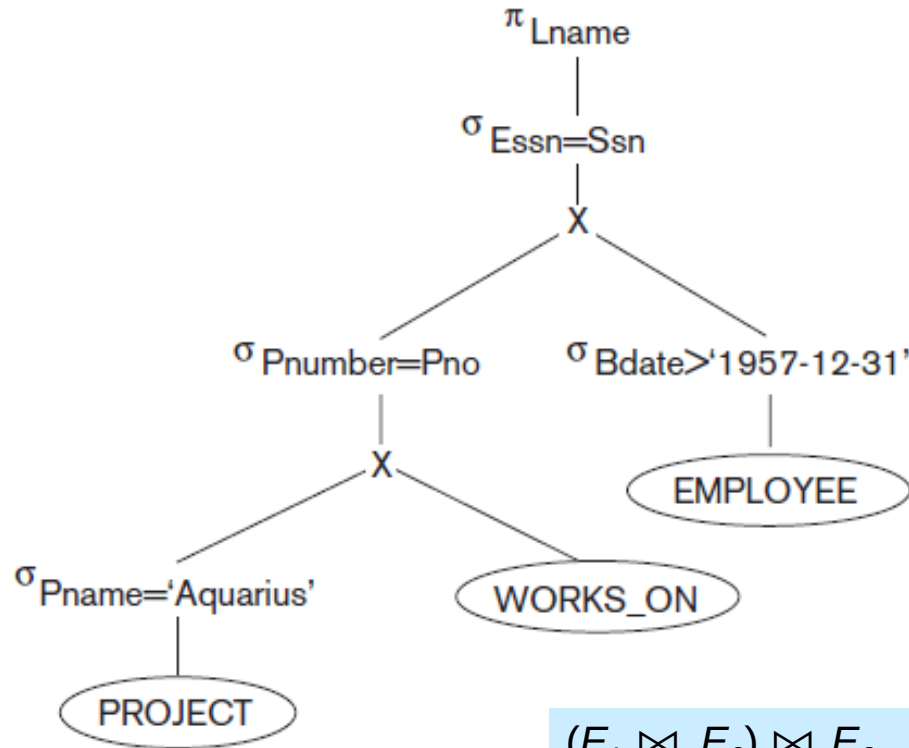


$$\sigma_{\theta_0 \wedge \theta} (E_1 \times E_2) \equiv \sigma_{\theta} ((\sigma_{\theta_0}(E_1)) \times E_2)$$

Steps in converting a query tree during heuristic optimization
(b) Moving SELECT operations down the query tree.

Query Transformation - Example 2 (cont'd.)

(c)



$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

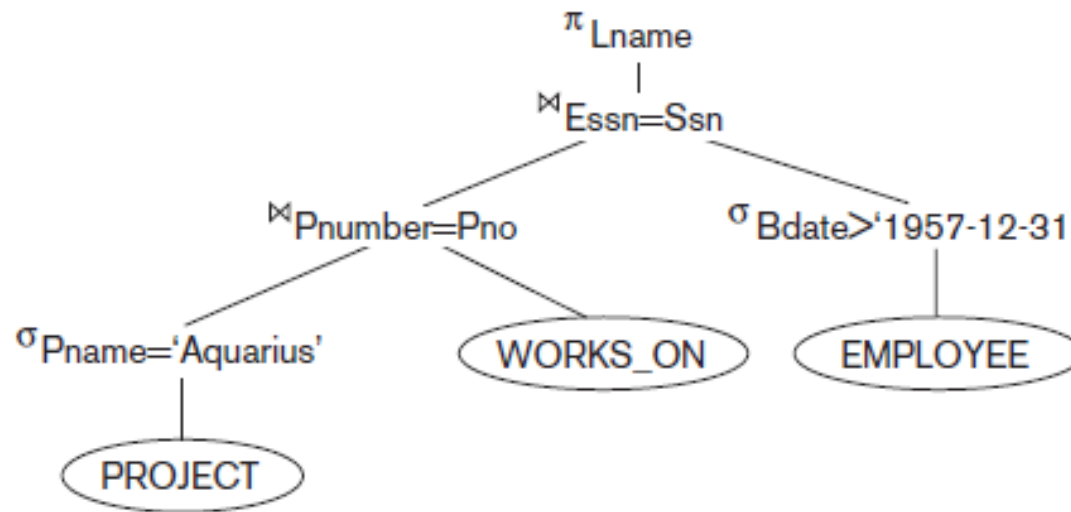
Steps in converting a query tree during heuristic optimization

(c) Applying the more restrictive SELECT operation first.



Query Transformation - Example 2 (cont'd.)

(d)



$$\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$$

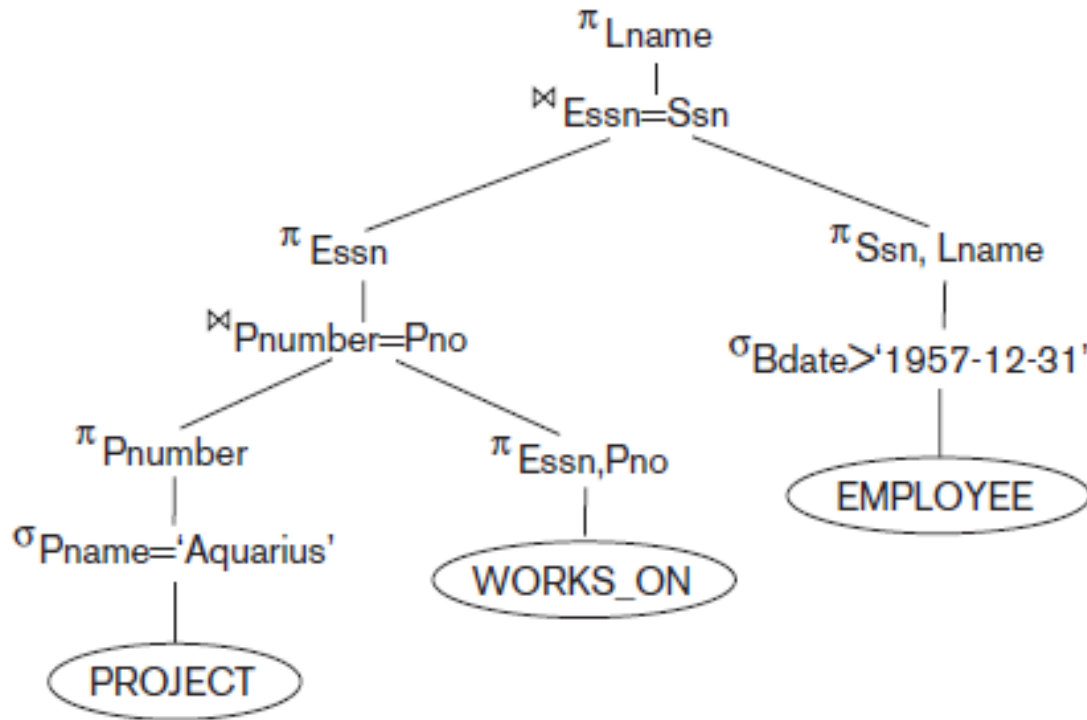
Steps in converting a query tree during heuristic optimization

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.



Query Transformation - Example 2 (cont'd.)

(e)



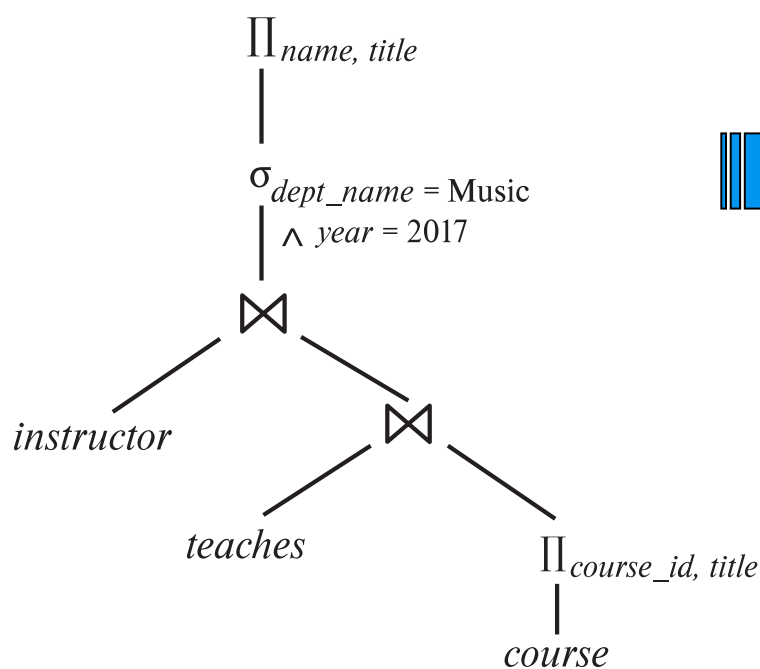
$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$

Steps in converting a query tree during heuristic optimization

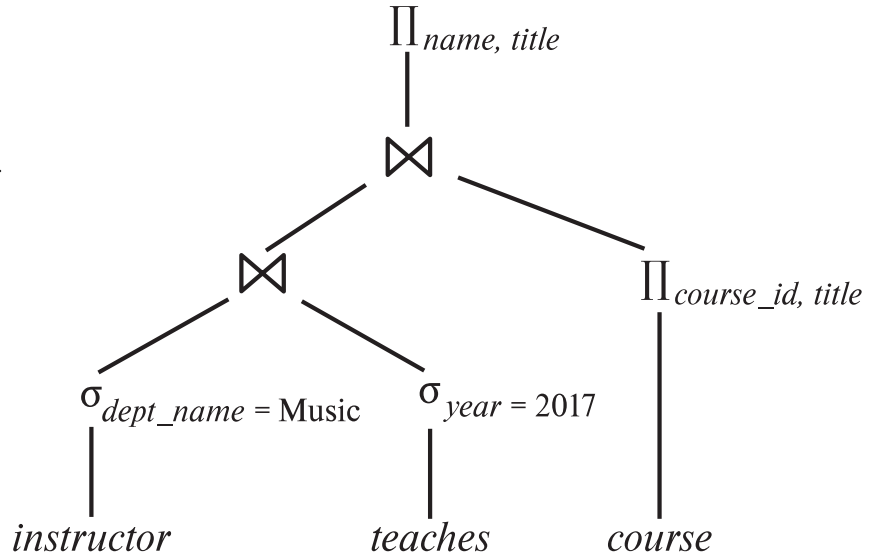
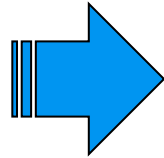
(e) Moving PROJECT operations down the query tree.



Query Transformation - Example 3



(a) Initial expression tree



(b) Tree after multiple transformations



Query Rewriting Rules Summary

Rule #	Name	Logic	Optimization Goal
1	Conjunctive Selection	$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$	Break complex filters into a sequence of simpler ones.
2	Commutative Selection	$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$	Reorder filters so the most "selective" (strictest) filter runs first.
3	Projection Omission	$\Pi_{L_1}(\Pi_{L_2}(E)) \equiv \Pi_{L_1}(E)$	If multiple projections exist, only the outermost (last) one matters.
4	Combine Select/Join	$\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$	Crucial: Never create a Cartesian Product if you can do a Join instead.
5	Commutative Join	$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$	Put the smaller table on the "inner" side to save memory.
6	Associative Join	$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$	Change the grouping of joins to keep intermediate results small.
7	Selection Pushdown	$\sigma_{\theta}(E_1 \bowtie E_2) \equiv (\sigma_{\theta}(E_1)) \bowtie E_2$	Filter rows early. Don't join data you're just going to delete later.
8	Projection Pushdown	$\Pi_L(E_1 \bowtie E_2) \equiv \Pi_{L_1}(E_1) \bowtie \Pi_{L_2}(E_2)$	Filter columns early. Keep the rows "thin" to save RAM.



Enumeration of Equivalent Expressions

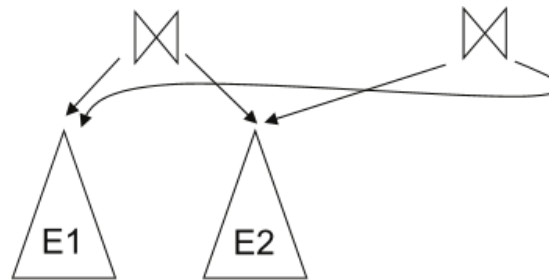
- ❑ Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression
- ❑ Can generate all equivalent expressions as follows:
 - ❑ Repeat
 - ▶ apply all applicable equivalence rules on every subexpression of every equivalent expression found so far
 - ▶ add newly generated expressions to the set of equivalent expressions

Until no new equivalent expressions are generated above
- ❑ The above approach is very expensive in space and time
 - ❑ Two approaches
 - ▶ Optimized plan generation based on transformation rules
 - ▶ Sharing of subexpressions



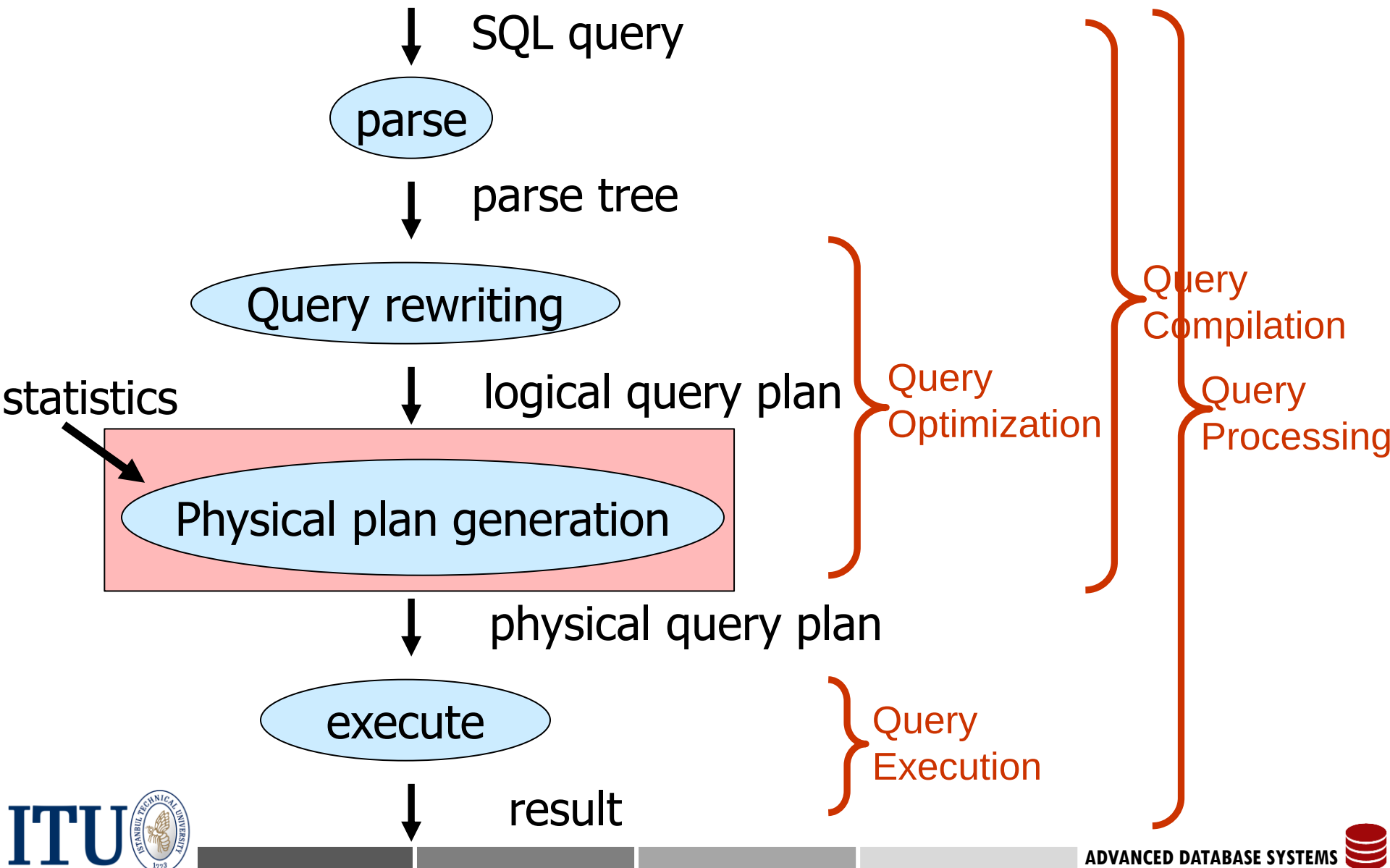
Implementing Transformation-Based Optimization

- Space requirements reduced by sharing common sub-expressions:
 - when an expression is generated from another expression by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers
 - ▶ e.g., when applying join commutativity



- Same sub-expression may get generated multiple times
 - ▶ Detect duplicate sub-expressions and share one copy
- Time requirements are reduced by not generating all expressions
 - Dynamic programming
 - ▶ We will study only the special case of dynamic programming for join order optimization

Overview of Query Processing



Enumeration of Alternative Plans

- **Task:** create a query execution plan for a single Select-project-join block (well, and aggregates).
- **Main principle:** some sort of search through the set of plans.
 - Assume some cost estimation model; more later.
- **Single-relation block case** (**select, project, aggregation**):
 - Each available access path is considered
 - ▶ the one with the least estimated cost is chosen.
 - Different operations are carried out together
 - ▶ e.g., if an index is used for a selection, projection is done for each retrieved tuple, and
 - ▶ the resulting tuples are *pipelined* into the aggregate computation.



Cost Estimation

- Cost of each operator
 - Computed as discussed in previous weeks
 - Need statistics of input relations
 - ▶ e.g., number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
 - Need to estimate statistics of expression results
 - To do so, we require additional statistics
 - ▶ e.g., number of distinct values for an attribute
- More on cost estimation later



Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - Choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. e.g.:
 - ▶ merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
 - ▶ nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.



Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Join Ordering Example (Cont.)

Consider the expression:

$$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course)$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{"Music"}}(instructor)$$

- However, the result of the first join is likely to be a large relation.
- Only a small fraction of the university's instructors are likely to be from the Music department
 - Better to compute first

$$\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches$$



Cost-Based Optimization

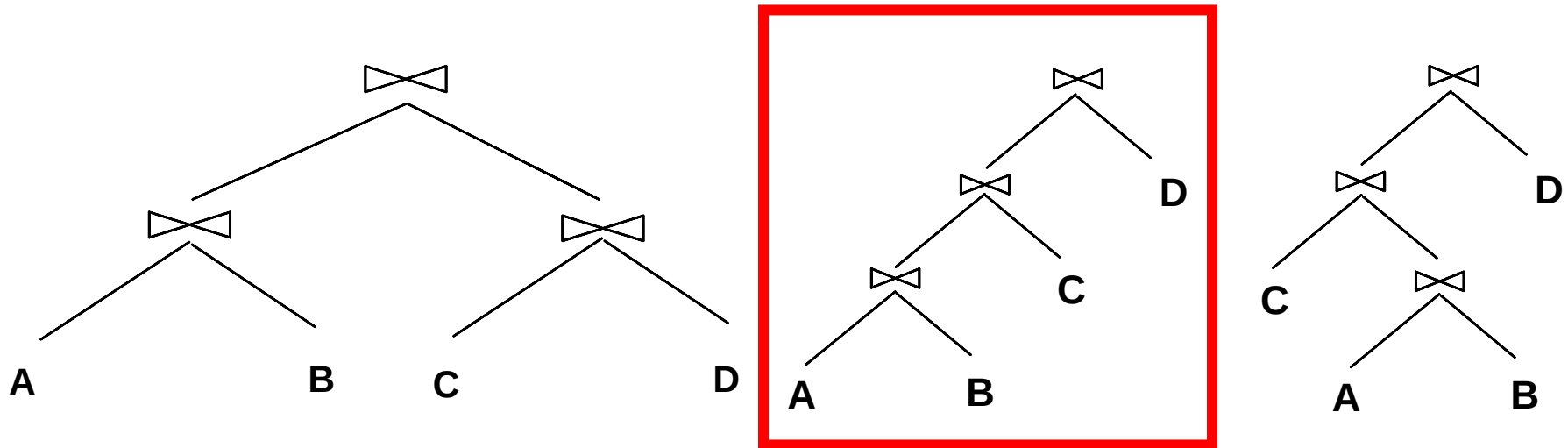
Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$.

- $(2(n-1))!/(n-1)!$ different join orders for the above expression.
 - With $n = 7$, the number is 665280,
 - With $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders.
 - Use dynamic programming!
 - The least-cost join order for any subset of $\{r_1, r_2, \dots, r_n\}$ is computed only once and stored for future use.



Queries Over Multiple Relations

- In principle, need to consider all possible join orderings:



- As the number of joins increases, the number of alternative plans grows rapidly; *we need to restrict the search space.*
- Most RDBMSs consider only left-deep join trees.
 - Left-deep trees allow us to generate **usually** *fully pipelined plans*:
 - ▶ Intermediate results not written to temporary files.
 - Not all left-deep trees are fully pipelined (e.g., SM join).

Cost Functions for the JOIN Operation (cont'd.)

- Multirelation queries and JOIN ordering choices
 - Left-deep join tree
 - Right-deep join tree
 - Bushy join tree

No. of Relations N	No. of Left-Deep Trees $N!$	No. of Bushy Shapes $S(N)$	No. of Bushy Trees $(2N - 2)! / (N - 1)!$
2	2	1	2
3	6	2	12
4	24	5	120
5	120	14	1,680
6	720	42	30,240
7	5,040	132	665,280



Dynamic Programming in Optimization

- To find best join tree for a set S of n relations:
 - Consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan.
 - Choose the cheapest of the 2^{n-1} alternatives.
 - Base case for recursion: single relation access plan
 - ▶ Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it!
 - ▶ Dynamic programming



Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S using selections on S and indices (if any) on S
  else
    for each non-empty subset S1 of S such that S1  $\neq$  S
      P1= findbestplan(S1)
      P2= findbestplan(S - S1)
      for each algorithm A for joining results of P1 and P2
        ... compute plan and cost of using A (see next slide) ..
      if cost < bestplan[S].cost
        bestplan[S].cost = cost
        bestplan[S].plan = plan;
    return bestplan[S]
```



Join Order Optimization Algorithm (cont.)

```
for each algorithm  $A$  for joining results of  $P1$  and  $P2$ 
  // For indexed-nested loops join, the outer could be  $P1$  or  $P2$ 
  // Similarly for hash-join, the build relation could be  $P1$  or  $P2$ 
  // We assume the alternatives are considered as separate algorithms
  if algorithm  $A$  is indexed nested loops
    Let  $P_i$  and  $P_o$  denote inner and outer inputs
    if  $P_i$  has a single relation  $r_i$  and  $r_i$  has an index on the join attribute
       $plan =$  “execute  $P_o.plan$ ; join results of  $P_o$  and  $r_i$  using  $A$ ”,
      with any selection conditions on  $P_i$  performed as part of
      the join condition
       $cost = P_o.cost + \text{cost of } A$ 
    else  $cost = \infty$ ; /* cannot use indexed nested loops join */
  else
     $plan =$  “execute  $P1.plan$ ; execute  $P2.plan$ ;
    join results of  $P1$  and  $P2$  using  $A$ ;”
     $cost = P1.cost + P2.cost + \text{cost of } A$ 
```

... See previous slide



Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
 - With $n = 10$, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations:
 - Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.
 - Modify optimization algorithm:
 - ▶ replace: “**for each** non-empty subset $S1$ of S such that $S1 \neq S$ ”
 - ▶ by: **for each** relation r in S
let $S1 = S - r$
- If only left-deep trees are considered, time complexity of finding best join order is $O(n 2^n)$
 - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n , generally < 10)



Enumeration of Left-Deep Plans

- Enumerated using N passes (if N relations joined):
 - **Pass 1:** Find best 1-relation plan for each relation.
 - **Pass 2:** Find best way to join result of each 1-relation plan (as outer) to another relation. (*All 2-relation plans.*)
 - **Pass N :** Find best way to join result of a $(N-1)$ -relation plan (as outer) to the N 'th relation. (*All N -relation plans.*)
- An $N-1$ way plan is not combined with an additional relation unless there is a join condition between them, unless all predicates in WHERE have been used up.
 - i.e., **avoid Cartesian products if possible.**
- For each subset of relations, retain only:
 - Cheapest plan overall, plus
 - Cheapest plan for each *interesting order* of the tuples.



Interesting Sort Orders

- Consider the expression $(r_1 \bowtie r_2) \bowtie r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could make a later operation (join/group by/order by) cheaper
 - Using merge-join to compute $r_1 \bowtie r_2$ may be costlier than hash join but generates result sorted on A
 - Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
- Not sufficient to find the best join order for each subset of the set of n given relations
 - Must find the best join order for each subset, **for each interesting sort order**
 - Simple extension of earlier dynamic programming algorithms
 - Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly



Enumeration of Plans (Contd.)

- ORDER BY, GROUP BY, aggregates etc. handled as a final step, using either an 'interestingly ordered' plan or an additional sorting operator.



Sailors:

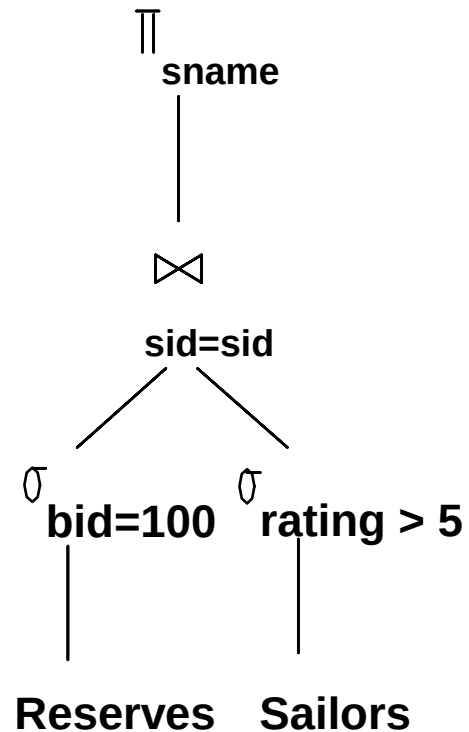
B+ tree on *rating*

Hash on *sid*

Reserves:

B+ tree on *bid*

Example



Pass 1:

- Sailors:** B+ tree matches *rating* > 5, and is probably cheapest. However, if this selection is expected to retrieve a lot of tuples, and index is unclustered, file scan may be cheaper.

- Still, B+ tree plan kept (tuples are in *rating* order).

- Reserves:** B+ tree on *bid* matches *bid*=100; cheapest.

Pass 2: how to join relations?

- e.g., **Reserves as outer:** Hash join could be used to get *Sailors* tuples that satisfy *sid* = outer tuple's *sid* value.



Heuristic Optimization

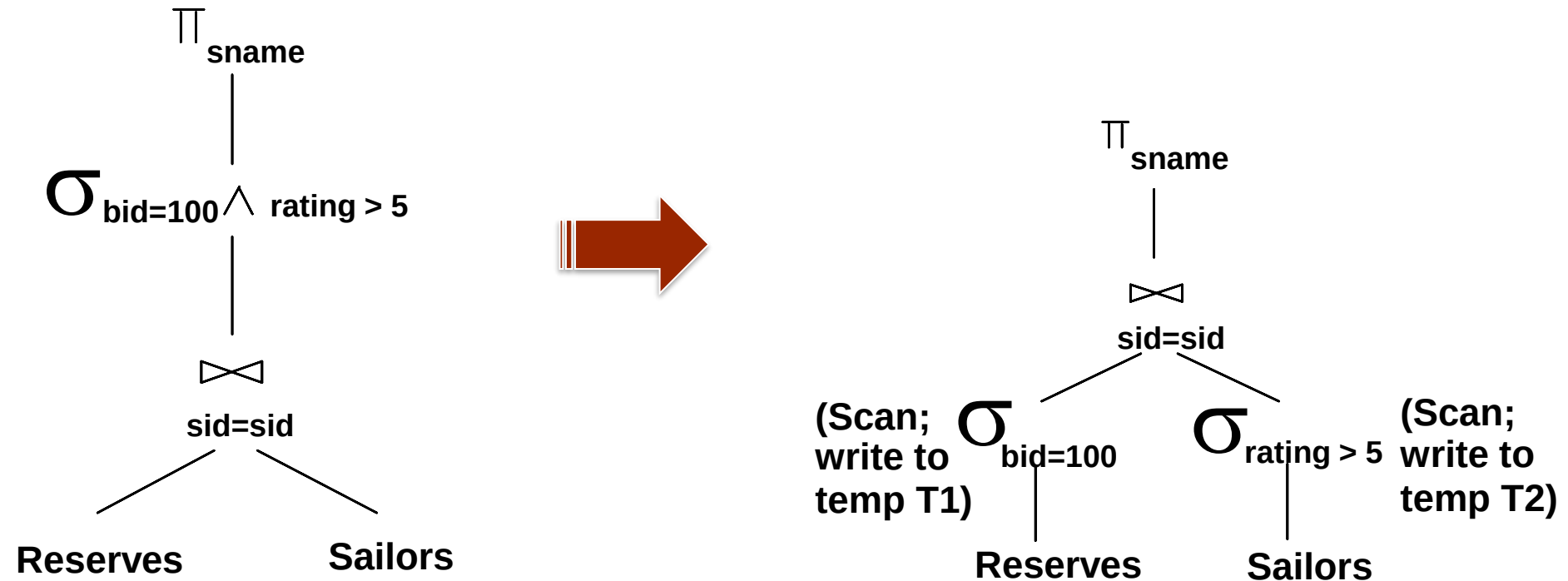
- ❑ Cost-based optimization is expensive, even with dynamic programming.
- ❑ Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- ❑ Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - ❑ Perform selection early (reduces the number of tuples)
 - ❑ Perform projection early (reduces the number of attributes)
 - ❑ Perform most restrictive selection and join operations (i.e., with smallest result size) before other similar operations.
- ❑ Some systems use only heuristics, others combine heuristics with partial cost-based optimization.



HEURISTICS FOR ALGEBRAIC OPTIMIZATION



Query Rewriting: Predicate Pushdown



The earlier we process selections, less tuples we need to manipulate higher up in the tree.

Query Rewrites: Predicate Pushdown (more complicated)

```
Select  bid, Max(age)
From    Reserves R, Sailors S
Where   R.sid=S.sid
GroupBy bid
Having  Max(age) > 40
```



```
Select  bid, Max(age)
From    Reserves R, Sailors S
Where   R.sid=S.sid and
        S.age > 40
GroupBy bid
Having  Max(age) > 40
```

- Advantage: the size of the join will be smaller.
- Requires transformation rules specific to the grouping/aggregation operators.
- Will it work if we replace Max by Min?



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age
```

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

*First, move
predicates up the
tree.*

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age and age < 20
```

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

*First, move
predicates up the
tree.*

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age
```

*Then, move them
down.*

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid and
 S.age < 20

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Structure of Query Optimizers

- Some query optimizers integrate heuristic selection and the generation of alternative access plans.
 - Frequently used approach
 - ▶ heuristic rewriting of nested block structure and aggregation
 - ▶ followed by cost-based join-order optimization for each block
 - Some optimizers (e.g. SQL Server) apply transformations to entire query and do not depend on block structure
- **Optimization cost budget** to stop optimization early (if cost of plan is less than cost of optimization)
- **Plan caching** to reuse previously computed plan if query is resubmitted
 - ▶ Even with different constants in query
- Even with the use of heuristics, cost-based query optimization imposes a substantial overhead.
 - But is worth it for expensive queries
 - Optimizers often use simple heuristics for very cheap queries, and perform exhaustive enumeration for more expensive queries

